Responsible AI Agents: A Research Paper Template

First Author[1] **Second Author**[2]

[1]Rein Protocol Foundation [2]Collaborating Institution
author@example.org

ABSTRACT

System behavior emerges across software, model, compiler, and runtime boundaries. This Rein Protocol Foundation template supports papers that examine those boundaries through an explicit threat model, an inspectable design, and controlled evaluation. The two-column layout includes a full-width architecture figure, compact tables, pseudocode, a code example, numeric citations, and artifact guidance. The document is instructional sample content, with no empirical security or performance claims. Replace every example with the study's own design, evidence, and limitations.

Keywords AI agents; evaluation; reproducibility; responsible AI

1 Introduction

Begin with a concrete workload or failure mode, identify the responsible component, and explain the observable consequence. Connect a learned model's behavior to the software and runtime that execute it.

Problem and gap. Specify what existing validation observes and misses. Preserving a reference implementation's property through a transformation is a different question from establishing that the reference was correct. Keep these claims separate.

Design insight. Describe the interface and oracle used to compare executions. Identify where a policy is enforced and which components it trusts. Protection principles provide context [1]; the paper must still supply a concrete model and supporting evidence.

The sample structure supports three kinds of contribution:

- A precisely scoped problem or threat model.
- A system, analysis, or defense with an explicit mechanism.
- An evaluation of effectiveness, overhead, and failure cases.

2 Scope and Threat Model

Assets and boundaries. Identify the protected property and trust boundaries. State whether the model, compiler, dependencies, runtime, and host are trusted. Distinguish data flow from control and privilege in diagrams.

Adversarial control. List what an adversary can read, modify, or query, with the available budget and feedback. For non-adversarial work, specify faults and workloads instead.

Algorithm 1 Illustrative discrepancy collection

Require: Programs P, Q, tests $\mathcal{T}$, threshold τ
Ensure: Findings F and execution errors E

$F \leftarrow \emptyset$; $E \leftarrow \emptyset$
for each $x \in \mathcal{T}$ **do**
  $(a, b, s) \leftarrow \text{RunPair}(P, Q, x)$
  if $s \neq \text{OK}$ **then**
    $E \leftarrow E \cup \{(x, s)\}$
  else if $d(a, b) > \tau$ **then**
    $F \leftarrow F \cup \{(x, a, b)\}$
  end if
end for
return (F, E)

Success and exclusions. Define success as an observable outcome. Explain exclusions such as physical access or privileged host compromise. Exclusions narrow the claim; they do not establish safety.

3 System Design

3.1 Inputs, outputs, and invariants

Let P be a reference program, $C(P)$ its transformed version, and $\mathcal{T}$ a finite test set. A comparison-based study might inspect

$$\Delta(x) = d\big(P(x), C(P)(x)\big), \qquad x \in \mathcal{T}, \tag{1}$$

where d is a domain-appropriate distance. State whether outputs are logits, labels, traces, or program states; these choices determine what the oracle can detect. A small value of Δ does not imply a general security guarantee, and a large value requires interpretation against the declared specification.

Definition 3.1 (Observed discrepancy)**.** For a declared threshold $\tau \geq 0$, an input x exhibits an observed discrepancy when $\Delta(x) > \tau$ under the fixed evaluation configuration.

3.2 Mechanism and design rationale

Figure 1 separates specification, construction, execution, and validation. Identify the new idea, each transformation's interface, and persistent state. For a heuristic, state its rationale and the experiment that tests its usefulness.

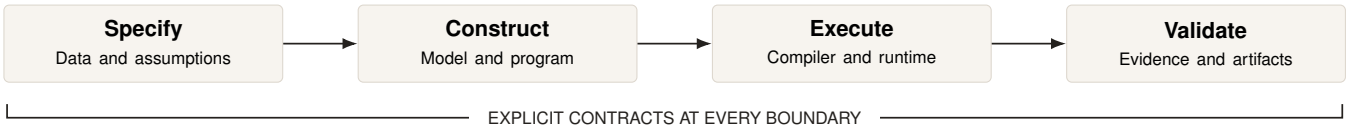


Figure 1. Illustrative architecture across the learning-to-execution pipeline. The full-width float demonstrates how a systems paper can show component interfaces without shrinking labels to column width.

Algorithm 1 separates discrepancies from failed executions. Distinguish crashes, timeouts, invalid inputs, and nondeterminism so aggregates do not mix incompatible outcomes.

3.3 Implementation details

Report the language, versions, environment, and configuration. Explain timeout handling, state resets, and failure recording. Place complete commands and configuration files in the artifact.

Table 1. Compact results skeleton. All cells are intentionally unfilled; dashes represent missing measurements.

System	Found	FP rate	Time (s)
Baseline A	—	—	—
Baseline B	—	—	—
MethodName	—	—	—
Ablation	—	—	—

4 Evaluation

4.1 Research questions and subjects

Tie each question to a capability: effectiveness measures the target behavior, overhead measures resources, and an ablation isolates a design decision's contribution.

Describe subject selection, version pinning, exclusions, and the unit of analysis. Repeated inputs from one model are not automatically independent subjects. Identify the population the conclusions concern.

4.2 Baselines, metrics, and controls

Use strong baselines and match budgets. Define every denominator, including the negative population for a false-positive rate. Explain labeling, deduplication, and how distinct root causes were determined.

Report workload shape, concurrency, warm-up, repetition, and hardware. Distinguish throughput from latency. Record model checkpoints, preprocessing, and precision; introduce learning concepts only as needed [2].

4.3 Analysis and failure cases

Use Table 1 to answer the research question. Match uncertainty to the unit of analysis. Explain whether gains span subjects or concern a small subset. Analyze failures that challenge the method's assumptions.

EVIDENCE BEFORE CLAIMS

Replace table placeholders with measured values. A discrepancy, a confirmed defect, and an exploitable vulnerability are different outcomes; label each according to the evidence collected.

5 Discussion and Limitations

Internal and construct validity. Explain how oracle errors, unstable execution, or configuration differences affect the conclusion. State the checks performed and residual uncertainty. For manual labels, describe the protocol and ambiguous cases.

External validity. Identify the versions, workloads, architectures, and deployments covered. A fixed benchmark does not establish production-wide coverage. Discuss the conditions and evidence supporting transfer.

Ethics and disclosure. Describe actual safeguards, affected parties, and disclosure status. For sensitive data or human participants, describe consent, access, and review. Include only actions performed.

6 Conclusion

Summarize the concrete problem, the mechanism, and the strongest supported finding. This sample demonstrates the REIN two-column layout and makes no empirical claims. A completed paper should end with the consequence of its evidence and the limits of its scope.

Acknowledgments

Replace with verified funding and assistance details. This section is hidden automatically in anonymous mode.

References

[1] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63 (9):1278–1308, 1975.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

A Artifact and Reproduction Guide

An artifact should connect each reported result to an executable command, configuration, and raw output. Record immutable source revisions and dependency versions. Document the time and resources needed for a small sanity check and for the full evaluation.

A.1 A minimal execution record

```
record = {
    "subject": "replace-with-id",
    "revision": "replace-with-commit",
    "seed": 0,
    "status": "ok",
    "raw_output": "results/run.json",
}
```

Listing 1. Illustrative record format; replace fields with the study's actual schema.

Use separate statuses for timeouts, crashes, invalid inputs, and completed runs. Preserve enough evidence to distinguish a harness failure from a failure of the system being studied.

A.2 Release manifest

- Source and configuration with explicit licenses.
- Data provenance, access rules, and preprocessing steps.
- Hardware and software environment specifications.
- Commands that regenerate each table and figure.
- Raw outputs, labeling records, and known limitations.

B Working with the REIN Template

Edit `paper-info-double.tex` for paper metadata and the files in `sections/double/` for content. Store reusable notation in `macros.tex`. The `reinpaper.cls` file controls both layouts; the two-column option switches the typeface, geometry, citation style, and title treatment together.

Use `figure` for a column-width illustration and `figure*` for an illustration spanning both columns. Full-width floats normally appear at the top of a later page. Refer to every figure in the text, and keep its labels readable at its final printed size.

B.1 Anonymous and grayscale modes

The `anonymous` class option removes the Foundation logo, author block, affiliation, emails, identifying running furniture, acknowledgments, and author metadata managed by the class. It does not inspect prose or third-party assets. Review links, self-references, and supplementary files before an anonymous submission.

The `monochrome` option changes the class colors to grayscale. The Foundation logo and external figures retain their original colors and must be checked separately.

B.2 Using an official venue style

Use this original Foundation style for manuscripts and preprints. Use the official venue template for conference submissions. Reuse section files and the bibliography, load the required packages, and replace class-specific commands. Remove identifying marks from anonymous manuscripts.